

# Quoridor Java 2025-2026

De la modélisation par graphes aux stratégies d'IA multi-paradigmes

# Présentation du Quoridor

Le Quoridor est un jeu de stratégie abstrait, déterministe et à information parfaite, conçu par Mirko Marchesi (1997). Il se joue sur un plateau carré de 9x9 cases, bien que le projet autorise des variantes allant de 3x3 à 15x15 tant que la taille reste impaire. Le jeu peut opposer deux ou quatre joueurs.



# Les règles du jeu

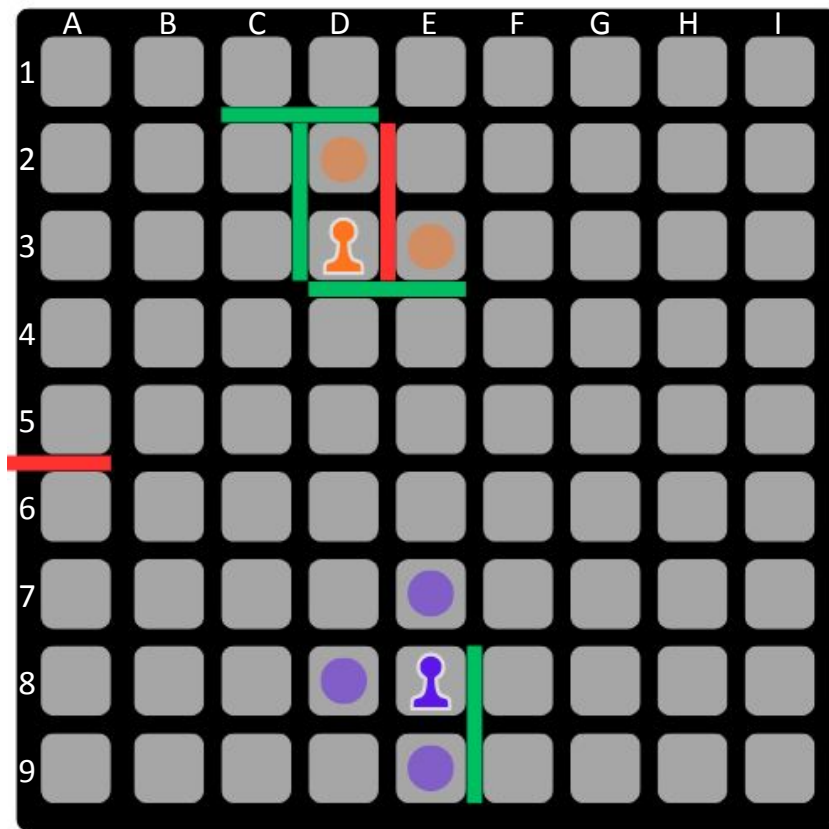
Chaque joueur dispose d'un pion et d'un nombre de murs. 10 à 2 joueurs, 5 à 4 joueurs.

Objectif : être le premier à atteindre la ligne opposée à sa ligne de départ.

À chaque tour, deux actions possibles :

- déplacer son pion d'une case (horizontalement ou verticalement)
- poser un mur pour bloquer la progression de ses adversaires.

La seule contrainte majeure est l'interdiction de bloquer totalement l'accès d'un joueur à sa ligne d'arrivée, un chemin libre doit toujours subsister.



# Cas particulier

Il est possible de sauter par dessus un autre joueur si une des cases autour de lui est libre et qu'on est juxtaposé à lui. 2 cas existent :

1e cas : Un joueur en bloque un autre. La case derrière lui est vide. Saut autorisé !

2nd cas : Un joueur en bloque un autre mais un mur est derrière lui. Il rebondit si une case est disponible. Saut en diagonal autorisé !



# Analyse de la complexité du jeu

## Complexité de l'espace d'états

Le Quoridor combine déplacements de pions et modifications topologiques (murs).

- **Positions des pions** :  $81 \times 80 = 6480$  configurations.
- **Placements de murs** : 128 positions théoriques (64 intersections x 2 orientations).
- **Formule de Mertens** :

$$S_f = \sum_{n=0}^{20} \left( \prod_{i=0}^{n-1} (128 - 4i) \right)$$

- **Estimation totale (Mertens, 2006)** :  $\sim 10^{42}$  (en multipliant la formule de Mertens par le nombre de configurations possibles)

## Dynamique et Arbre de jeu (G)

L'explosion combinatoire rend l'exploration exhaustive impossible.

- **Facteur de branchement** :  $\sim 60$  (vs 35 aux échecs).
- **Profondeur moyenne** : 100 demi-coups (50 tours).
- **Complexité de l'arbre** :  $G = b^d = 60^{\{100\}} \approx 10^{177}$  (Complexité des échecs :  $10^{123}$ )

# Ce qui existe aujourd'hui

## **Adaptations numériques**

- Automatisation de l'arbitrage : Plateformes web (Board Game Arena) et supports mobiles.
- Respect des règles : Mise en œuvre logicielle des contraintes de mouvement et de pose de murs.

## **Littérature scientifique de référence**

- Thèses fondatrices : Travaux de Glendenning (2005) et Mertens (2006).
- Contributions : Formalisation de la notation algébrique et développement des premières heuristiques d'évaluation pour les agents intelligents.

## **Analyse algorithmique contemporaine**

- Recherche active : Projets comme Quoridor AI et analyses stratégiques récentes (Quoridor Story, M. Lloyd, 2022).
- Objectif : Résolution algorithmique et optimisation des moteurs de décision.

# Modélisation par Graphe

**Structure** : Graphe non orienté  $G = (V, E)$ .

**Sommets (V)** : 81 cases (indexées de 0 à 80).

**Arêtes (E)** : Transitions possibles entre cases adjacentes.

**Choix technique : Liste d'adjacence.**

- Densité faible (max 4 voisins/sommet).
- Efficacité mémoire :  $O(V + E)$ .
- Accès rapide :  $O(1)$ .
- Murs gérés comme des contraintes de transition (invalidation d'arêtes).

# Algorithmes de Recherche de Chemin

## **BFS (Parcours en largeur) :**

- Calcul de la distance minimale vers la ligne d'arrivée.
- Utilisé pour la fonction d'évaluation de l'IA.

## **Validation des coups :**

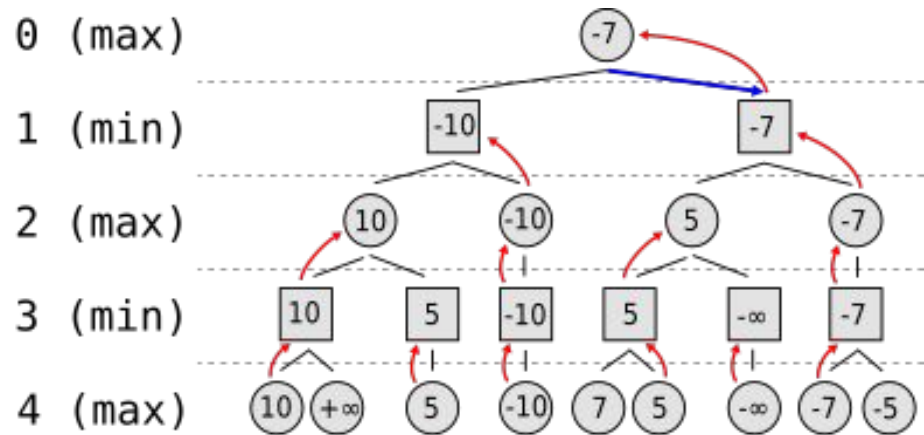
- Vérification de la connectivité via les composantes connexes.
- Interdiction de poser un mur si le joueur n'appartient pas à la même composante connexe qu'une case de la ligne d'arrivée.(blocage total).

# IA - Minimax et Élagage Alpha-Beta

**Principe :** Exploration de l'arbre de décision en anticipant les coups adverses.

**Optimisation Alpha-Bêta :**  
Réduction drastique des branches explorées sans perte de précision.

**Fonction d'évaluation :** Basée sur la distance BFS et la gestion du stock de murs restant.



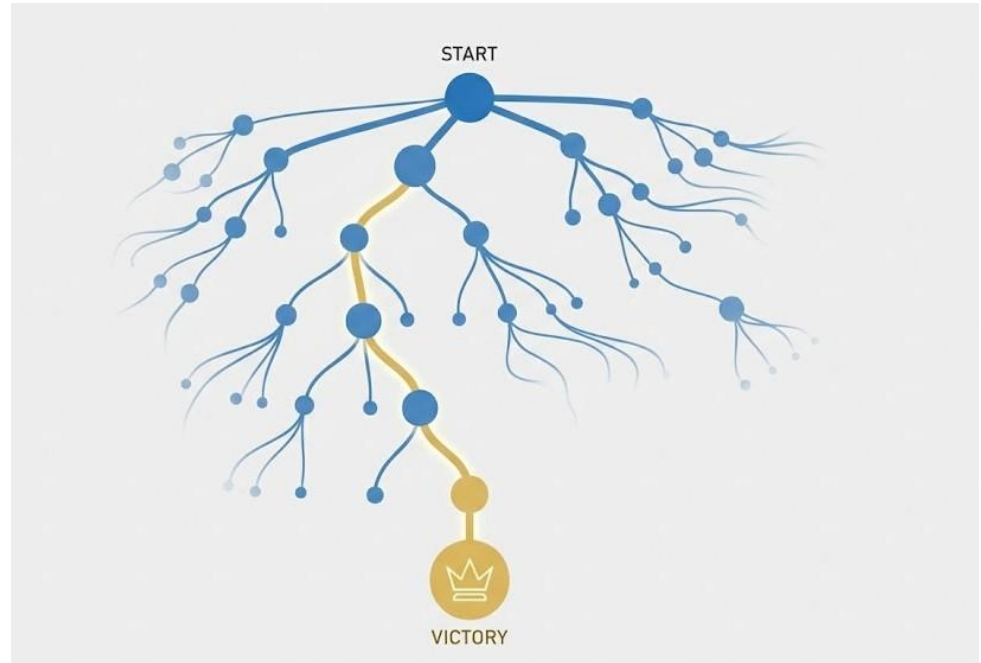
# IA - Monte Carlo Tree Search (MCTS)

**La Sélection** : L'IA choisit le chemin qui semble le plus prometteur parmi ceux qu'elle connaît déjà.

**L'Expansion** : Arrivée au bout de sa connaissance, elle ajoute un nouveau coup à tester.

**La Simulation** : Elle termine la partie au hasard (très vite) pour voir si ce nouveau coup mène à une victoire.

**La Mise à jour** : Elle attribue une note au chemin : positive si elle gagne, négative si elle perd, pour affiner ses futurs choix.



# IA - Apprentissage Automatique

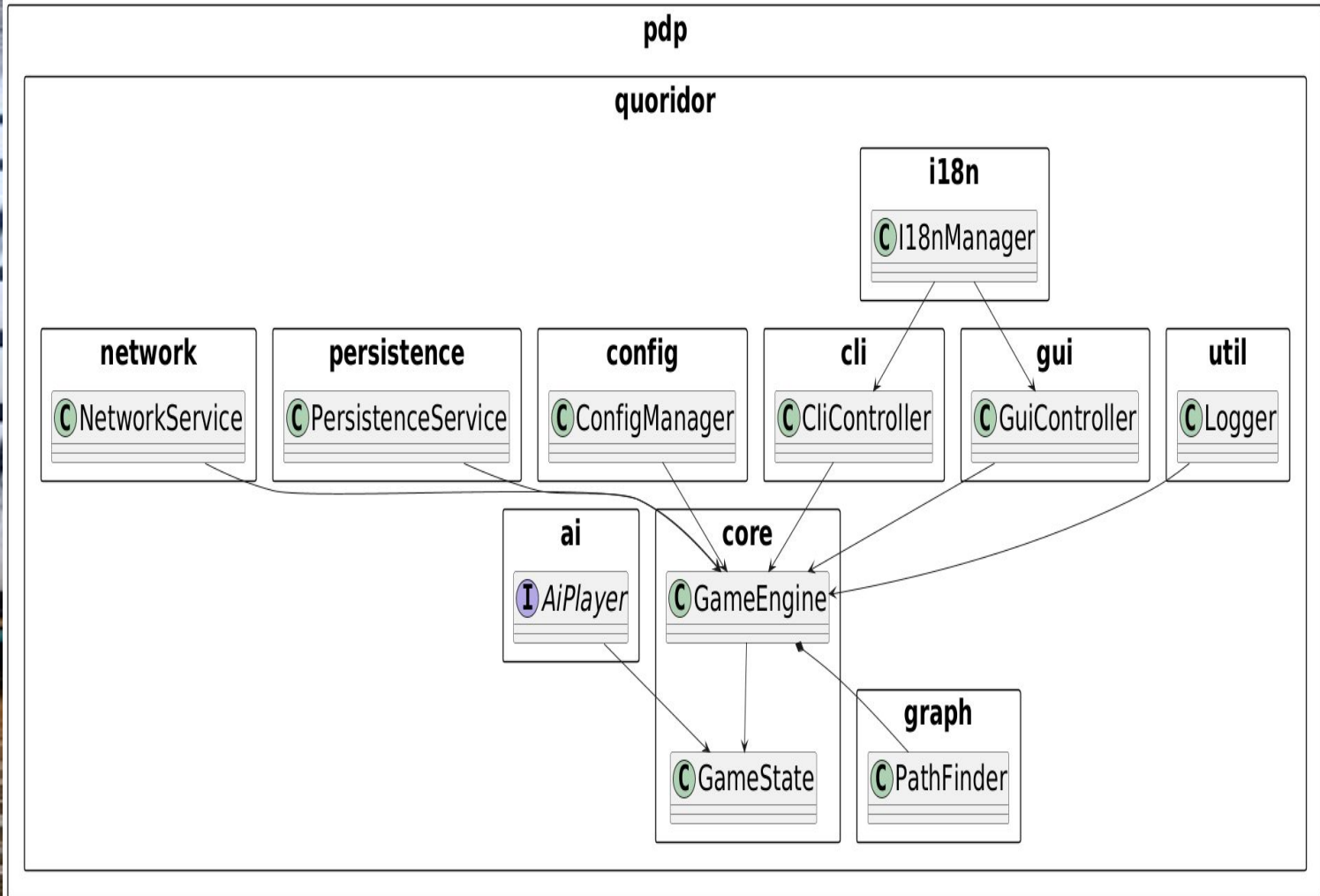
**Optimisation** : Amélioration de la sélection des nœuds dans le MCTS. Plutôt que d'être **effectuée** au hasard, la sélection des coups dépend désormais de probabilités définies par le modèle.

**Modèle** : Régression logistique ou arbre de décision.

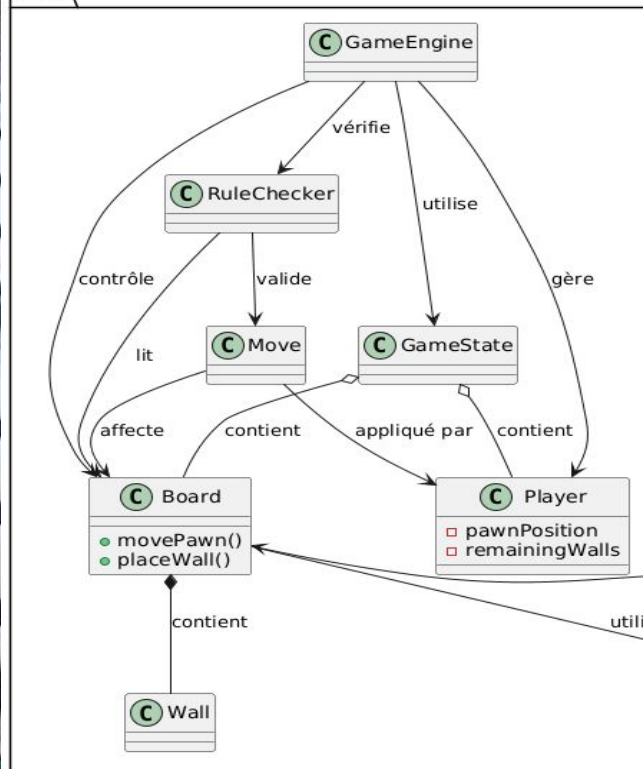
**Entraînement** : Basé sur des historiques de parties pour identifier statistiquement les configurations gagnantes.

**Caractéristiques d'évaluation (Features)** : Distances minimales à l'arrivée pour les deux joueurs et leurs nombres de murs respectifs.

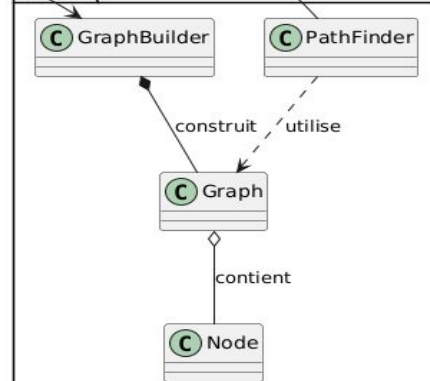
# Présentation de l'architecture



core



graph



# Merci de votre attention

Abdeldjabar Ismail Abdraman  
Cengiz Vasseur  
Joris Douillet  
Khadidja Ahmat Hassan  
Matthieu Pourageaud